CHAPTER 7

Building Full Vue Apps

CLI · Router · State Management

From a single HTML file to a full Single Page Application

3-Hour Lecture and Lab Session

Learning Outcomes

By the end of this chapter, students will be able to...

1. **Scaffold a Vue project**

   Create a new Vue app using create-vue / Vite with the standard project structure.

2. **Author Single File Components**

   Build .vue files with template, script, and style blocks.

3. **Communicate between components**

   Pass data down with props, raise events with emit, and compose with slots.

4. **Add multi-page navigation**

   Configure Vue Router with routes, router-link, and router-view.

5. **Share state across components**

   Use Pinia to create stores and keep app-wide state in one place.

From CDN to Build Tools

Why we outgrow a single HTML file

Chapter 6 — CDN

- Good for learning & small pages
- One big HTML file
- No components (just templates)
- No hot reload
- No build pipeline
- No module imports

Chapter 7 — Build Tools (Vite)

- Multiple .vue files (Single File Components)
- Hot Module Replacement (instant feedback)
- ES modules, npm packages
- TypeScript, CSS modules, preprocessors
- Production build: minified, tree-shaken
- Dev-server optimised via Vite

Creating a Vue Project

create-vue scaffolding powered by Vite

Requirements: Node.js 18+ and a terminal. Every new Vue project starts with:

```
$ npm create vue@latest

? Project name:        my-app
? Add Vue Router?      Yes
? Add Pinia?           Yes
? Add TypeScript?      No
? Add ESLint?          Yes

$ cd my-app
$ npm install
$ npm run dev

  VITE v5.x  ready in 312 ms
  ➜  Local:   http://localhost:5173/
```

Why Vite?

- Instant dev server (ESM, no bundling)
- Hot Module Replacement (< 50 ms)
- Fast builds via Rollup
- Official recommendation from Vue team
- Replaces the older Vue CLI

Project Structure

The typical layout of a Vite-powered Vue project

```
my-app/
├── node_modules/
├── public/                  # static assets
│   └── favicon.ico
├── src/                     # application source
│   ├── assets/              # images, global css
│   ├── components/          # reusable .vue components
│   ├── views/               # route-level .vue pages
│   ├── router/index.js      # Vue Router config
│   ├── stores/              # Pinia stores
│   ├── App.vue              # root component
│   └── main.js              # entry point
├── index.html               # app shell (just one!)
├── package.json
└── vite.config.js
```

src/main.js

Creates the Vue app, registers plugins, mounts to #app.

src/App.vue

Root component — the starting template of your SPA.

src/components/

Small reusable UI pieces (buttons, cards, lists).

src/views/

Page-level components mapped to routes.

src/router/

Vue Router — all routes in one place.

src/stores/

Pinia stores — shared state.

Single File Components (.vue)

Three blocks: <template>, <script>, <style>

```
<!-- HelloCard.vue -->
<template>
  <div class="card">
    <h2>{{ title }}</h2>
    <p>Clicked {{ count }} times</p>
    <button @click="count++">+1</button>
  </div>
</template>

<script setup>
import { ref } from 'vue'
defineProps({
  title: { type: String, default: 'Hello!' }
})
const count = ref(0)
</script>

<style scoped>
.card { padding: 1rem;
        border: 2px solid #42B883;
        border-radius: 12px; }
</style>
```

`<template>`

HTML-like markup with directives & interpolation.

`<script setup>`

Composition API with a simpler syntax.

`<style scoped>`

CSS that only affects this component.

Using a Component

Import it, then use it as a tag

```
<!-- App.vue -->
<template>
  <HelloCard title="Welcome" />
  <HelloCard title="Students" />
  <HelloCard />
</template>

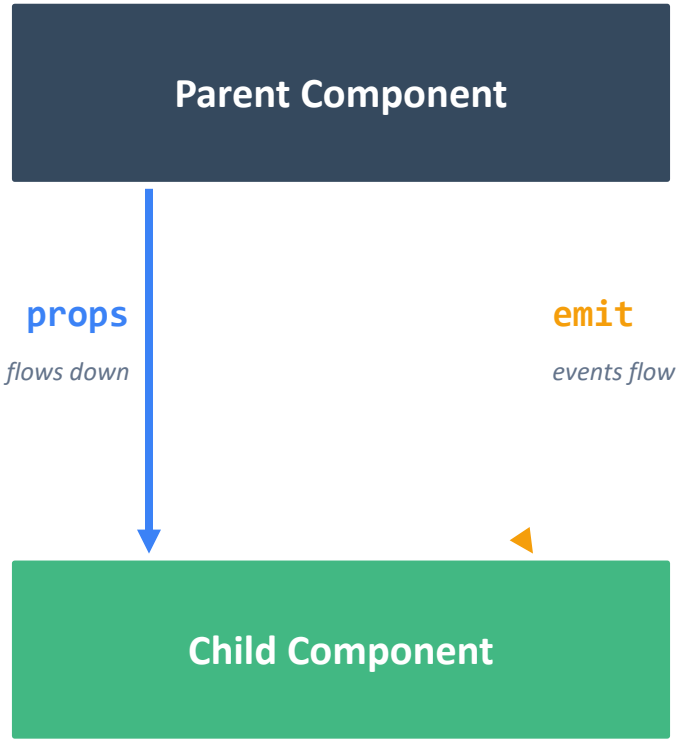
<script setup>
import HelloCard from './components/HelloCard.vue'
</script>
```

Rules

- Import local components
  — no registration needed in <script setup>
- PascalCase in templates (<HelloCard/>)
- Or kebab-case (<hello-card/>)
- Self-closing tags are allowed
- Each component has isolated scope
  (state is local unless props/emits/store)

Component Communication

How parents and children talk to each other



Props
Parent passes data down to the child

Emit
Child tells the parent something happened

Slot
Parent injects HTML into a hole in the child

Pinia
For data shared across many components,
or deeply nested hierarchies

Why not just lift state up?
Works for small apps, but becomes "prop drilling" at scale.

Props — Parent → Child

Declare with defineProps(), pass in the template

```
<!-- Child: UserCard.vue -->
<template>
  <div>
    <h2>{{ name }}</h2>
    <p>{{ email }}</p>
  </div>
</template>

<script setup>
// declare & validate props
const props = defineProps({
  name:  { type: String, required: true },
  email: { type: String, default: 'n/a' }
})
</script>
```

```
<!-- Parent: App.vue -->
<template>
  <UserCard
    :name="user.name"
    :email="user.email"
  />
</template>

<script setup>
import UserCard from './UserCard.vue'
import { reactive } from 'vue'

const user = reactive({
  name:  'Farah',
  email: 'farah@example.com'
})
</script>
```

Events — Child → Parent

defineEmits() in the child, @event in the parent

```
<!-- Child: SaveButton.vue -->
<template>
  <button @click="onClick">Save</button>
</template>

<script setup>
const emit = defineEmits(['saved'])

function onClick() {
  // do work, then notify parent
  emit('saved', { ok: true, at: Date.now() })
}
</script>
```

```
<!-- Parent: App.vue -->
<template>
  <SaveButton @saved="onSaved" />
  <p v-if="status">Saved!</p>
</template>

<script setup>
import SaveButton from './SaveButton.vue'
import { ref } from 'vue'

const status = ref(false)
function onSaved(payload) {
  console.log(payload)
  status.value = true
}
</script>
```

Slots — Content Injection

Let the parent decide what goes inside

```
<!-- Child: Card.vue -->
<template>
  <div class="card">
    <header>
      <slot name="title">Default title</slot>
    </header>

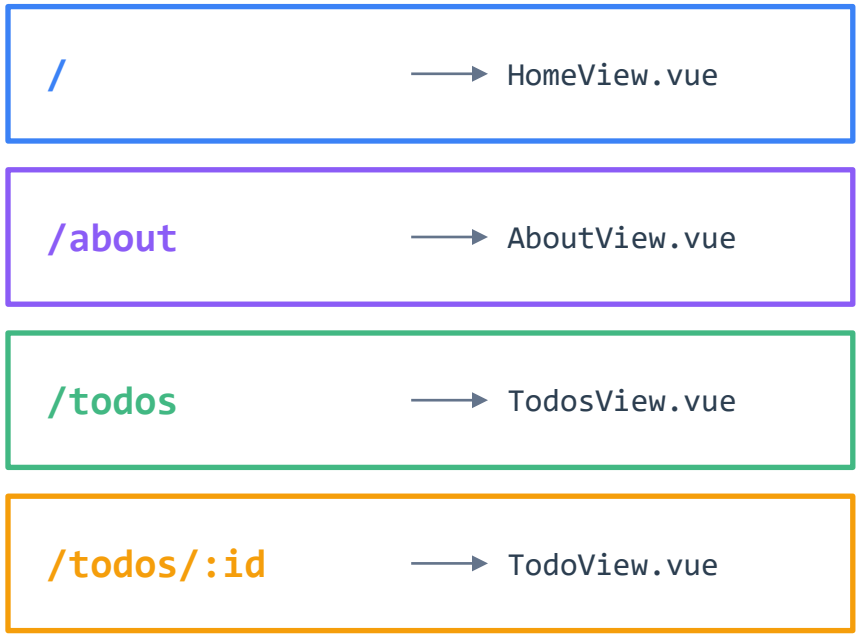
    <div class="body">
      <slot /> <!-- default slot -->
    </div>
  </div>
</template>
```

```
<!-- Parent -->
<Card>
  <template #title>
    Welcome back, Ali!
  </template>

  <p>Here are your tasks for today.</p>
  <ul>
    <li>Read Chapter 7</li>
    <li>Build the lab</li>
  </ul>
</Card>
```

Vue Router — Multi-Page SPAs

One HTML page, many 'pages' via URL



Key ideas

- Map URLs to components
- Client-side routing — no page reload
- Built-in <router-link> & <router-view>
- Dynamic params with :id
- Programmatic navigation (router.push)
- Nested routes, guards, lazy loading

Configuring the Router

src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView  from '../views/HomeView.vue'
import TodosView from '../views/TodosView.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/',       name: 'home',  component: HomeView  },
    { path: '/todos',  name: 'todos', component: TodosView },
    { path: '/todos/:id',
      name: 'todo',
      component: () => import('../views/TodoView.vue') // lazy
    },
    { path: '/:pathMatch(.*)*', component: () => import('../views/NotFound.vue') }
  ]
})

export default router
```

Register in main.js: app.use(router)

<router-link> and <router-view>

Navigation tags provided by Vue Router

```
<!-- App.vue -->
<template>
  <nav>
    <router-link to="/">Home</router-link>
    <router-link :to="{ name: 'todos' }">
      Todos
    </router-link>
    <router-link to="/about">About</router-link>
  </nav>

  <!-- active route component renders here -->
  <router-view />
</template>
```

`<router-link>`

Renders an <a> tag with client-side routing.
Auto class 'router-link-active' when URL matches.

`<router-view>`

The placeholder where the matched component is rendered.

Programmatic

```
router.push('/todos')
router.back()
```

Dynamic Routes & Params

URLs like /todos/42 — read :id inside the component

```
// router config
{ path: '/todos/:id', name: 'todo', component: TodoView }

// TodoView.vue
<template>
  <h2>Todo #{{ id }}</h2>
  <router-link to="/todos">Back</router-link>
</template>

<script setup>
import { useRoute } from 'vue-router'

const route = useRoute()
const id = route.params.id   // "42"

// navigate programmatically
// import { useRouter } from 'vue-router'
// const router = useRouter()
// router.push({ name: 'todo', params: { id: 7 } })
</script>
```

Why State Management?

When props & events stop being enough

Problem: prop drilling

```
App.vue
 └─ Layout.vue
     └─ Sidebar.vue
         └─ UserMenu.vue (needs user)
```

Passing user through 4 layers just to read it once is painful and fragile.

Solution: a shared store

Store

A central object with reactive state + actions.

Any component can read or modify it.

Vue's official library for this is **Pinia.**

Pinia — Creating a Store

src/stores/todos.js

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useTodosStore = defineStore('todos', () => {
  const items = ref([])                                  // state

  const doneCount = computed(() =>                       // getter
    items.value.filter(t => t.done).length)

  function add(text) {                                   // actions
    items.value.push({ id: Date.now(), text, done: false })
  }
  function remove(id) {
    items.value = items.value.filter(t => t.id !== id)
  }
  function toggle(id) {
    const t = items.value.find(t => t.id === id)
    if (t) t.done = !t.done
  }

  return { items, doneCount, add, remove, toggle }
})
```

Using a Pinia Store

Call the composable in any component

TEMPLATE

```
<!-- TodosView.vue -->
<template>
  <input v-model="text"
         @keyup.enter="submit">
  <button @click="submit">Add</button>

  <ul>
    <li v-for="t in store.items" :key="t.id">
      <input type="checkbox"
             :checked="t.done"
             @change="store.toggle(t.id)">
      {{ t.text }}
      <button @click="store.remove(t.id)">x</button>
    </li>
  </ul>

  <p>{{ store.doneCount }} /
     {{ store.items.length }} done</p>
</template>
```

SCRIPT

```
<script setup>
import { ref } from 'vue'
import { useTodosStore }
  from '../stores/todos'

const store = useTodosStore()
const text  = ref('')

function submit() {
  const t = text.value.trim()
  if (!t) return
  store.add(t)
  text.value = ''
}
</script>

// store.items, store.doneCount,
// store.add(), store.remove(),
// store.toggle()  — all reactive!
```

Wiring Everything in main.js

Install plugins on the Vue app

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'

import './assets/main.css'

const app = createApp(App)

app.use(router)       // Vue Router
app.use(createPinia()) // State management

app.mount('#app')
```

Remember

- app.use() installs plugins
- Order rarely matters here
- mount() must come LAST
- index.html has one
- Vite handles the rest

LAB Brief — Todo List SPA

Build a full Vue app with routing and a Pinia store

You will build

- Scaffold with npm create vue@latest
- 3 views: Home, Todos, Todo detail
- Pinia store for the todos list
- Custom TodoItem component (props + emit)
- Router with dynamic :id params
- Add / toggle / delete / view detail
- Style it nicely (your choice)

Steps

1. **npm create vue@latest (pick Router + Pinia)**
2. **Run npm run dev**
3. **Define todos store in src/stores/**
4. **Create TodoItem.vue with props + emit**
5. **Build TodosView.vue — list + add form**
6. **Build TodoDetailView.vue using :id param**
7. **Wire routes in router/index.js**
8. **Polish styles & submit**

Full guide in the lab hand-out.

Summary — Key Takeaways

What to remember from Chapter 7

1. **Vite scaffolds full Vue projects**

   npm create vue@latest gives you a batteries-included app.

2. **SFCs are the unit of composition**

   template + script + scoped style in a single .vue file.

3. **Props down, events up**

   The 1-way data flow keeps components predictable.

4. **Vue Router = multi-page SPA**

   Map URLs to components; navigate with <router-link>.

5. **Pinia solves shared state**

   Stores provide reactive state + actions to any component.

References & Next Steps

Vue 3 guide: https://vuejs.org/guide/introduction.html

Vite: https://vitejs.dev

Vue Router: https://router.vuejs.org

Pinia: https://pinia.vuejs.org

Style guide: https://vuejs.org/style-guide/

What's next?

Deploy your app (Netlify / Vercel), explore composables, add a backend API.

Terima kasih — Thank you!